Manual API Testing compact cheatsheet

STEP 1 Manual API testing course

1.1 Manual API Testing: Basics to Expert

What an API is and why we test it An API (Application Programming Interface) is a set of rules that lets one program ask another program for data or for an action.

What a web service is A web service is an API that one program calls over a network using web standards such as HTTP, XML or JSON. Two systems use it to exchange data no matter which language or operating system each one runs on.

REST in depth REST was defined by Roy Fielding in his doctoral dissertation in 2000. He was also one of the authors of the HTTP specification. An API that follows the REST constraints is called RESTful .

Kinds of APIs and where they sit in a system

How HTTP carries a request Most APIs you will test use HTTP (Hypertext Transfer Protocol), and almost always HTTPS , which is HTTP inside an encrypted TLS connection. HTTP is a request and response protocol. The client sends one request, the server sends back one response.

HTTP methods, safety and idempotency The method tells the server what you want to do with the resource. Two properties matter a lot for testing. A safe method only reads and never changes data. An idempotent method gives the same server state whether you send it once or ten times.

Status codes The three digit status code is the first thing you check. The first digit gives the family.

Headers, cookies and content negotiation Headers carry metadata about the request or the response. Header names are case insensitive, so content-type and Content-Type are the same header.

JSON, XML and other payloads JSON (JavaScript Object Notation) is the default format for modern APIs. It has six value types: string, number, boolean, null, object and array. Keys must be in double quotes, and trailing commas are not allowed.

REST, SOAP, GraphQL, gRPC, WebSocket and webhooks REST is covered in depth in Part 3 . The quick reminders below are the naming habits you will check most often.

Tools: Postman, curl, Bruno and friends

Seeing real traffic: DevTools, HAR files and proxies Documentation tells you what an API should receive. Real traffic tells you what the app actually sends. Many of the best API bugs are found by watching the calls a web page or a mobile app makes while you click around, then replaying them with small changes.

Reading API docs and OpenAPI Your test oracle is the contract. For REST APIs this is usually an OpenAPI document (formerly called Swagger) in YAML or JSON. It lists every path, method, parameter, request body, response, status code and security scheme.

Authentication and authorization Authentication answers who you are. Authorization answers what you are allowed to do. A missing or bad identity should give 401. A known identity without permission should give 403 (or 404 if the API hides that the resource exists).

Types of API testing and when to use each "API testing" is not one activity. It is a family of test types, and each one answers a different question. A hospital gives a good picture. A quick pulse check at the door is a smoke test. A full blood report is functional testing.

Designing API test cases The test design techniques from manual testing all apply to APIs. The difference is that you control every byte of the request, so you can reach cases the UI never allows.

What to check in every response A 200 status alone proves very little. A response can be 200 and still wrong. Check these layers every time.

Variables, chaining and end to end flows Real flows need data from earlier responses. You log in, save the token, create an order, save its id, then read, update and delete that same order. This is called request chaining .

Pagination, filtering and sorting No API returns a million rows at once. Lists come in pages. There are three common styles.

Rate limits, caching and idempotency keys APIs limit how many requests a client can make, to protect the service and to stop brute force attacks. When you exceed the limit you should get 429 Too Many Requests with a Retry-After header.

Error handling and Problem Details Good errors tell the client what went wrong and how to fix it, without leaking internals. The standard format is Problem Details for HTTP APIs , RFC 9457 (2023), which replaced RFC 7807. It uses the media type application/problem+json .

File uploads and downloads Profile photos, resumes on a job site, invoices in a banking app and medical reports in a health app all travel through APIs. Files bring their own bugs: huge sizes, wrong types, broken names and security holes.

Dates, money, text and language edge cases Some of the most famous software bugs had nothing to do with clever logic. They were about a clock, a currency or a name. Airline systems have shown flights landing before they took off because of time zones.

Concurrency, race conditions and double clicks Picture the last train ticket on a festival weekend. Two people in two cities press "Book" in the same second.

Checking side effects: database, queues, emails and logs A response of 201 Created tells you what the API claims happened. It does not prove it.

HTTPS, TLS certificates, CORS and HTTP versions Before your request reaches any business logic, it passes through a few lower layers: encryption, certificates and browser safety rules. When these break, the whole app breaks, often with confusing errors.

API security testing: the OWASP API Top 10 The OWASP API Security Top 10 (2023 edition) is the standard list of the most common API risks. Manual testers find many of these with nothing more than Postman and curiosity. Only test systems you are allowed to test.

Performance testing for APIs Even manual testers should note response times and payload sizes, and know the vocabulary of load testing.

Async APIs: webhooks, polling and queues Some work takes too long for one request, such as generating a report or processing a video. The API answers 202 Accepted with a job id, and the client either polls a status endpoint or waits for a webhook.

Real world scenario walkthroughs Here is how API testing looks on three real kinds of product. Pick a scenario to step through the calls, then read the test ideas a senior tester would bring to it.

Contract testing and versioning When ten teams depend on one API, a small change such as renaming a field can break apps that nobody on the provider team knows about. Contract testing catches this before release.

Mocks, service virtualization and test data You often need to test an API whose dependencies are not ready, are expensive (a real payment provider), or cannot be made to fail on demand. A mock server pretends to be that dependency.

Security tools and fuzzing: ZAP, Burp Suite and Schemathesis Manual thinking finds the clever bugs. Tools find the many small ones a person would never have time to try. Fuzzing means sending large numbers of unexpected, random or malformed inputs and watching for crashes, 500 errors, slow responses or data leaks.

Testing GraphQL, gRPC, WebSocket and SSE gRPC has its own status codes such as OK , INVALID_ARGUMENT , NOT_FOUND , PERMISSION_DENIED , UNAUTHENTICATED , RESOURCE_EXHAUSTED and DEADLINE_EXCEEDED .

API tests in CI, monitoring and observability Manual exploration finds the bugs. Saved collections stop them from coming back. A mature team runs its API collection on every pull request and on a schedule against live environments.

Testing AI and LLM APIs and MCP servers More and more products call large language model (LLM) APIs, or expose their own AI endpoints. These APIs behave differently from classic ones, and testers need new techniques.

Latest standards and trends API standards keep moving. These are the ones worth knowing about today (checked in October 2026). Some are final standards, and some are drafts that tools already support.

An API test strategy and great bug reports Always include the exact request (a curl command is ideal), the full response with status and body, the environment, the time in UTC and the request id. Remove real tokens and personal data before you paste.

EXAM "Is REST a protocol?" is a favourite interview trap. No. REST is an architectural style. HTTP is the protocol most REST APIs use. SOAP is a protocol.

TRAP A GET that changes data, such as GET `/orders/5/cancel`, is a design bug. Browsers, crawlers and caches may call GET links on their own and cancel real orders. Report it.

EXAM Idempotent does not mean the same response. The first `DELETE /orders/5` returns 204 and the second returns 404. The SERVER STATE is the same after both (order 5 is gone), so DELETE is still idempotent.

TRAP A HAR file can contain passwords, cookies, tokens and personal data. Chrome now strips cookies and Authorization headers from exported HAR files by default, but other tools may not. Before you attach a HAR to a ticket, open it in a text editor or a sanitizer tool and remove secrets.

TRAP Watch for responses that wrap errors inside a 200, such as `{"success": false, "error": "Invalid id"}`. Clients and monitors that only check the status will treat failures as success. Report it as a design issue.

EXAM Offset pagination with a changing data set is a classic interview question. If a new item is inserted at the top while you read page 1, the last item of page 1 moves to page 2 and you see it twice. Cursor pagination avoids this.

TRAP After a race test, always check the data, not just the status codes. Two 201 responses might be correct (two different items), while one 201 and one 500 might still have left half finished data behind.

TRAP Security testing on systems you do not own or have no written permission to test can be illegal. Use your team's QA environment, or practice on deliberately vulnerable apps such as OWASP crAPI and OWASP Juice Shop.

TRAP Scanners and fuzzers send a lot of traffic and can create, change or delete data. Run them only against environments you own or have written permission to test, never against production unless the team has planned it, and tell the team before a big run.